Git & GitHub

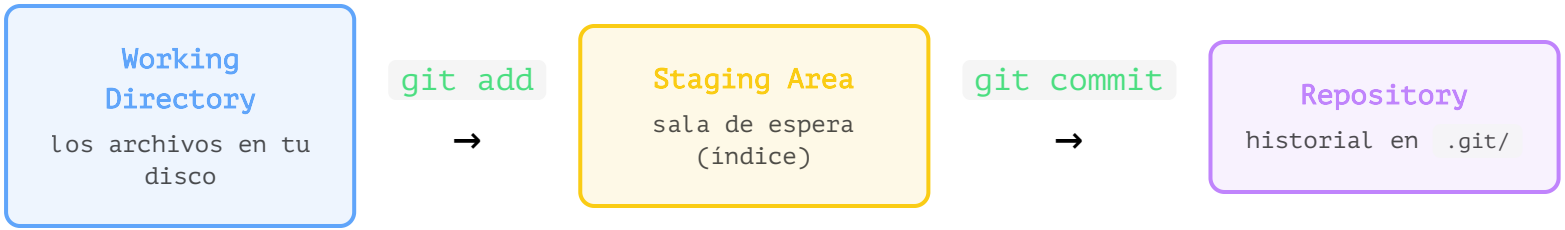
Ingeniería de Aplicaciones Web

2026

1. Conceptos Core de Git

Las tres áreas de trabajo

Git organiza el trabajo en tres zonas bien diferenciadas:



Working Directory
Los archivos tal como están en tu disco.

Staging Area
Preparás los cambios que van al próximo commit.

Repository
Base de datos de snapshots (vive en `.git/`).

💡 **Concepto clave:** Git no guarda diferencias entre archivos, guarda *snapshots* completos del estado del proyecto en cada commit.

Commits como snapshots

Cada commit contiene:

- Un **snapshot** del proyecto en ese momento
- Un **mensaje** descriptivo
- Referencia al commit **padre** (o padres, en merge)
- **Metadatos**: autor, fecha, hash SHA-1

```
A ——— B ——— C ——— D
                  ▲
                HEAD
```

```
commit d4f8a2c
Author: Ana García
Date: Thu Mar 26 2026

Add user authentication
```

HEAD

`HEAD` es un puntero que indica **dónde estás parado ahora**.

- Normalmente apunta a la **rama activa**
- La rama activa apunta al **último commit** de esa rama
- Cuando hacés `git checkout`, HEAD se mueve

```
main ——— A ——— B ——— C
                     ▲
                   main
                    HEAD
```

```
# Después de git checkout feature:

main    ——— A ——— B ——— C
feature                  ——— D
                             ▲
                         feature
                          HEAD
```

Comandos esenciales

```
1   git init                    # Inicializa un repositorio
2   git status                  # Estado del working directory y staging area
3   git diff                    # Diferencias en working directory (sin stagear)
4   git add [archivo]           # Agrega archivo al staging area
5   git add .                   # Agrega todos los cambios
6   git commit -m "mensaje"     # Crea un commit con los cambios en staging
7   git log                     # Historial de commits
8   git log --oneline --graph   # Historial compacto con grafo de ramas
9   git diff                    # Diferencias en working directory
10  git diff --staged           # Diferencias en staging area
```

2. Branching y Merging

¿Qué es una rama?

Una rama es simplemente **un puntero con nombre que apunta a un commit**.

- Crear una rama es **barato y rápido**
- Solo se crea un archivo de texto con un hash
- No hay copias de archivos

> 💡 En otros VCS crear una rama era costoso. En Git es instantáneo.

```
# Estado inicial
main ——— A ——— B ——— C
                     ▲
                  main / HEAD
```

```
# Después de git checkout -b feature
main    ——— A ——— B ——— C
                        ▲
                 main / feature / HEAD
```

```
# Después de un commit en feature
main    ——— A ——— B ——— C
                        \
feature                  D ——— E
                                ▲
                         feature / HEAD
```

Comandos de branching

```
1   git branch                        # Lista ramas locales
2   git branch [nombre]               # Crea una rama nueva
3   git checkout [nombre]             # Cambia a esa rama
4   git checkout -b <nombre>          # Crea y cambia en un paso
5   git branch -d <nombre>            # Elimina una rama (solo si fue mergeada)
```

Flujo típico:

```
1   # Desde develop, crear nueva feature
2   git checkout develop
3   git checkout -b feature/login
4
5   # ... trabajar y commitear ...
6
7   git checkout develop
8   git merge feature/login
```

Consejo moderno:

`git switch` es el comando nuevo para cambiar ramas:

```
1   git switch develop
2   git switch -c feature/login    # -c = create
```

Más explícito que `checkout` (que hace muchas cosas).

Tipos de Merge

Fast-forward

Rama destino **sin commits nuevos** — Git solo mueve el puntero.

```
Antes:
  main:    A — B
                \
  feature:       C — D
```

```
Después:
  main:    A — B — C — D
                        ▲
                      main
```

Three-way

Ambas ramas tienen commits — Git crea un merge commit con dos padres.

```
Antes:
  main:    A — B — C
                \
  feature:       D — E
```

```
Después:
  main:    A — B — C ——— M
                \         /
  feature:       D ——— E
```

M es el **merge commit**, tiene dos padres: C y E.

Resolución de conflictos

Si dos ramas modifican la **misma línea** de un archivo, Git no puede decidir por sí solo:

```
<<<<<<< HEAD
  return user.getName();
=======
  return user.getFullName();
>>>>>>> feature/login
```

Paso 1
Editar el archivo y dejar el código correcto (elegir uno o combinar ambos).

Paso 2
`git add <archivo>`
Marcar el archivo como resuelto.

Paso 3
`git commit`
Finalizar el merge con un commit.

Git Flow

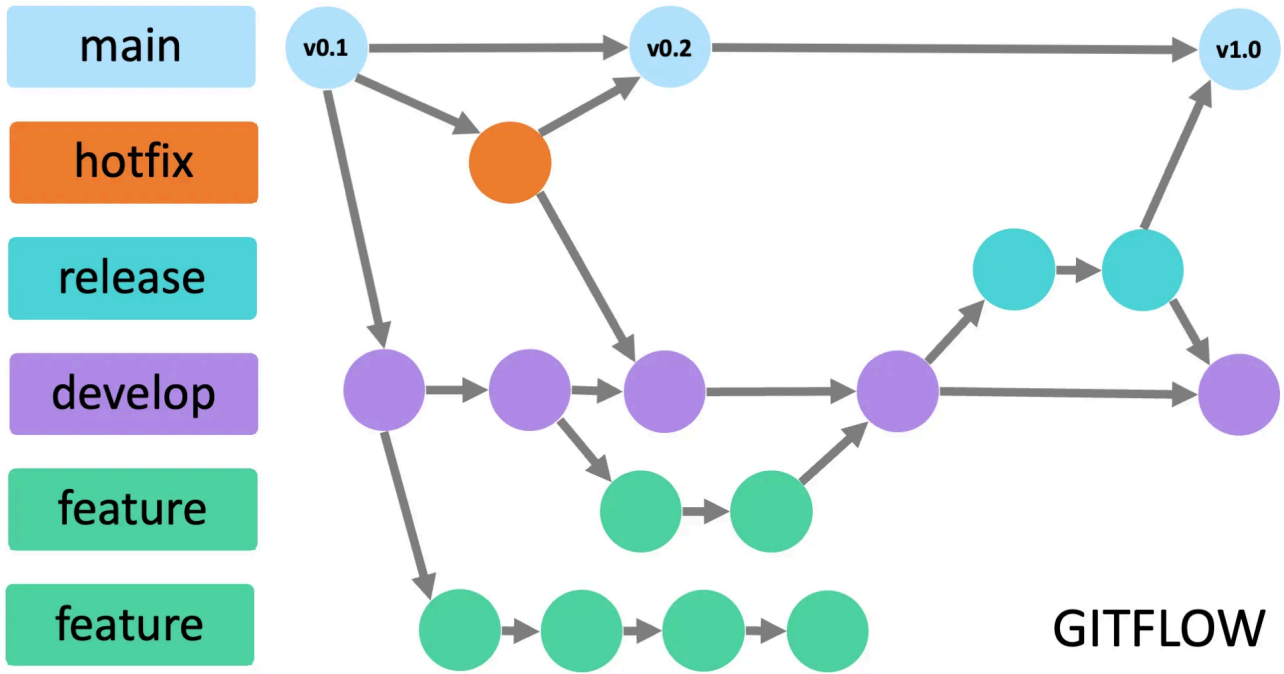
Modelo de branching para proyectos con releases

nvie.com/posts/a-successful-git-branching-model

Las cinco ramas de Git Flow

Rama	Propósito	Sale de	Se mergea en
`main`	Código en **producción**. Siempre estable.	—	—
`develop`	**Integración** continua de features.	`main`	`main` (via release)
`feature/*`	Desarrollo de una **funcionalidad**.	`develop`	`develop`
`release/*`	**Preparación** de un nuevo release.	`develop`	`main` + `develop`
`hotfix/*`	**Fix urgente** en producción.	`main`	`main` + `develop`

Git Flow — Diagrama



Flujo: Feature

```
# 1. Crear desde develop
git checkout develop
git checkout -b feature/login

# 2. Desarrollar y commitear
git add .
git commit -m "Add login form"
git commit -m "Add auth validation"

# 3. Mergear de vuelta
git checkout develop
git merge --no-ff feature/login

# 4. Limpiar
git branch -d feature/login
```

`--no-ff` fuerza un merge commit aunque sea posible fast-forward. Mantiene la trazabilidad de la feature.

Flujo: Release

```
# 1. Crear rama cuando develop está listo para salir
git checkout develop
git checkout -b release/1.2.0

# 2. Solo bug fixes menores y bump de versión
#    NO se agregan features nuevas aquí
git commit -m "Bump version to 1.2.0"
git commit -m "Fix minor formatting issue"

# 3. Finalizar: mergear a main y a develop
git checkout main
git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m "Release 1.2.0"

git checkout develop
git merge --no-ff release/1.2.0

git branch -d release/1.2.0
```

Flujo: Hotfix

```
# 1. Bug crítico en producción
git checkout main
git checkout -b hotfix/fix-crash-login

# 2. Corregir el bug
git add .
git commit -m "Fix null pointer on login"

# 3. Mergear a main Y develop
git checkout main
git merge --no-ff hotfix/fix-crash-login
git tag -a v1.2.1 -m "Hotfix 1.2.1"

git checkout develop
git merge --no-ff hotfix/fix-crash-login

# 4. Limpiar
git branch -d hotfix/fix-crash-login
```

¿Cuándo usar Git Flow?

✅ Git Flow es ideal cuando...

- Proyectos con **versiones numeradas** y releases planificados
- Equipos **medianos o grandes**
- Hay un ciclo claro de **QA antes de producción**
- Múltiples versiones en producción simultáneamente

⚡ Alternativa: Trunk-based

Solo `main` + feature branches de corta vida.

- Mejor para equipos **pequeños**
- Requiere **CI/CD maduro**
- Deployments frecuentes (varios por día)

En esta materia usaremos **Git Flow**.

3. GitHub: La Capa de Colaboración

Remotes

Un *remote* es una **copia del repositorio** alojada en otro servidor (GitHub, GitLab, etc.).

```
1   git remote -v                  # Ver remotes configurados
2   git clone [url]                # Clona un repo remoto (crea origin automáticamente)
3   git push origin [rama]         # Sube una rama al remote
4   git pull origin [rama]         # Baja cambios y los mergea
5   git fetch origin               # Baja cambios sin mergear (más seguro)
```

`git pull = git fetch + git merge`

Conveniente pero hace todo de golpe.

Recomendado: usar `fetch` primero para ver qué cambios llegaron antes de integrarlos.

```
1   git fetch origin
2   git log origin/develop  # revisar
3   git merge origin/develop
```

Forks vs. Branches

	Branch	Fork
Vive en	El mismo repositorio	Una copia propia del repositorio
Uso típico	Trabajo en equipo en el mismo proyecto	Contribuir a un proyecto ajeno
Permisos	Necesitás acceso al repo	No necesitás acceso al repo original

Branch → para trabajo del equipo en el mismo repo.

```
mi-repo/
  main
  develop
  feature/login    ← todos trabajan acá
```

Fork → para contribuir a proyectos open source.

```
facebook/react  (repo original)
    ↓ fork
ana/react       (tu copia)
  feature/new-hook  ← trabajás acá
    ↓ Pull Request
facebook/react  (propuesta de merge)
```

Pull Requests (PR)

Un PR es una **solicitud de merge con revisión de código** incorporada.

Flujo típico:

1. Crear rama `feature/X` desde `develop`
2. Pushear la rama a GitHub
3. Abrir un PR hacia `develop` en GitHub
4. Compañeros **revisan, comentan, aprueban**
5. Se **mergea** el PR (historial preservado)

Los PR permiten:

- Discutir cambios antes de mergear
- Hacer **comentarios línea por línea**
- Mantener registro de **decisiones técnicas**
- Ejecutar **checks automáticos** (CI)
- Requerir aprobaciones antes de mergear

Los PR son la herramienta central de colaboración en GitHub y en el mundo profesional.

GitHub Flow — documentación oficial

Issues & GitHub Actions

Issues

Permiten rastrear bugs, features y tareas.

```
# Referenciar un issue desde un commit:
git commit -m "Fix crash en login, closes #42"
```

- Se pueden **asignar** a personas
- Se pueden **etiquetar** (bug, feature, etc.)
- Se **cierran automáticamente** al mergear un PR que los referencia

GitHub Actions

Automatiza tareas en respuesta a eventos del repo.

```yaml
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm test
```

Lo veremos en detalle más adelante. Lo usaremos para **CI/CD**.

4. Flujo de Trabajo en el Curso

Estructura y convenciones

Repositorios

- Cada proyecto tendrá su repositorio en **GitHub**
- Modelo de branching: **Git Flow**
- Ramas `main` y `develop` protegidas

Convenciones de nombres

```
feature/<descripcion-corta>
release/<version>
hotfix/<descripcion>
```

Ejemplos:

```
feature/autenticacion-usuario
release/1.0.0
hotfix/fix-login-null
```

Proceso de entrega

1. Desarrollar usando **Git Flow**: features en `feature/*`, integradas a `develop`
2. Crear una rama `release/x.x.x` desde `develop` y finalizarla
3. Mergear el release a `main` y **tagear** con `vx.x.x`
4. Abrir un **Pull Request** desde `release/x.x.x` hacia `entrega`
5. El PR es la entrega — debe tener una **descripción clara** de qué se implementó

⚠️ No se acepta entrega sin tag de release en `main` ni sin el PR hacia `entrega`.

Ejercicio para casa

learngitbranching.js.org

Completar al menos los primeros ejercicios de la sección "**Secuencia principal**" (Main).

Visualización interactiva de commits y ramas

Practica los comandos en un entorno seguro

En la próxima clase comentamos las dudas que surgieron

Resumen

Concepto	Para recordar
Commit	Snapshot del proyecto, no un diff
Branch	Puntero liviano a un commit
Merge	Integrar el historial de dos ramas
Git Flow	`main` / `develop` / `feature` / `release` / `hotfix`
Remote	Copia del repo en otro servidor
Pull Request	Solicitud de merge con revisión de código

📸

Git = snapshots
No diffs, fotos completas

🌿

Branches = punteros
Baratos y poderosos

🤝

PRs = colaboración
Revisión antes de mergear

¿Preguntas?

Git & GitHub — Ingeniería de Aplicaciones Web · 2026